ASSESSMENT TOOL 2 – REPORT

NAME : RAGAVI S

REGISTER NO : 192565027

COURSE CODE : CSA1717

COURSE NAME : ARTIFICIAL INTELLIGENCE

QUESTION 1: Constraint Satisfaction Problem (Doctor Shift Scheduling)

1. AIM

To solve a doctor shift scheduling problem using Backtracking Search under a given set of hard constraints and derive a valid shift assignment.

2. OBJECTIVE

- To model the doctor assignment problem as a Constraint Satisfaction Problem (CSP) consisting of Variables, Domains, and Constraints.
- To apply the Backtracking Search algorithm step-by-step, showing intermediate constraint checks and pruning steps.
- To identify the final valid schedule that satisfies all domain and unary/binary constraints.

3. THEORY

A Constraint Satisfaction Problem (CSP) is defined by a set of variables X = {X1, X2, ..., Xn}, a set of domains D = {D1, D2, ..., Dn} for each variable, and a set of constraints C specifying allowable combinations of values.

Backtracking Search is a depth-first search approach tailored for CSPs. It assigns values to one variable at a time and backtracks immediately when a variable has no legal values left due to constraint violations.

Problem Formulation :

- **Variables:** D1, D2, D3 (Doctors)

- **Domains:** {Morning (M), Afternoon (A), Night (N)} for each doctor
- **Constraint 1:** D1 != Night (Unary constraint)
- **Constraint 2:** D2 < D3 in terms of shift order (Morning < Afternoon < Night)
- **Constraint 3:** All shifts are distinct: D1 != D2, D1 != D3, D2 != D3 (One doctor per shift)
- **Constraint 4:** D3 != Morning (Unary constraint)
- **Constraint 5:** Every doctor must be assigned exactly one shift

4. ALGORITHM (Backtracking Search for CSP)

Algorithm 1: Backtracking Search Procedure

Algorithm: Backtracking-Search(CSP)
Input: A constraint satisfaction problem containing variables, domains, and constraints.
Output: A complete consistent assignment or failure.

Procedure:
1. Start with an empty assignment set.
2. Call Recursive-Backtracking(assignment, CSP).
3. In Recursive-Backtracking:
   a. Base Case: If the assignment contains all variables, return the completed assignment.
   b. Variable Selection: Pick an unassigned variable Xi using a selection heuristic or default order.
   c. Domain Loop: For each value v in the domain of Xi:
      i. Check if assigning Xi = v satisfies all constraints given the current partial assignment.
      ii. If consistent, add Xi = v to the assignment.
      iii. Recursively invoke Recursive-Backtracking(assignment, CSP).
      iv. If the recursive call returns a success assignment, return it immediately.
      v. Otherwise, remove Xi = v from the assignment (backtrack) and try the next value v.
   d. Failure Return: If no value in the domain is consistent, return failure.

5. INTERMEDIATE STEPS & RESULT

Step-by-Step Execution:

- **Step 1: Reduced Domains (Unary Filtering) -** D1 in {M, A} (since D1 != Night)
  - D2 in {M, A, N}
  - D3 in {A, N} (since D3 != Morning)
- **Step 2: Branching & Constraint Checking -** Assign D1 = M. Valid so far. Remaining available shifts for others: {A, N}.
  - Try D2 = M -> VIOLATION (Constraint iii: D1 != D2).

- Try D2 = A -> Check Constraint ii (D2 < D3): Since D2 = A, D3 must be strictly later than A, so D3 = N.
- Check Constraint iii (D1 != D2): D1 = M, D2 = A (Valid).

- **Step 3: Completing Assignment -** Assign D3 = N.
  - Check Constraint iv (D3 != M): Valid.
  - Check Constraint ii (D2 < D3): D2 = A < D3 = N (Valid).
  - Check Constraint iii (All distinct): D1 = M, D2 = A, D3 = N (Valid).
  - Check Constraint v (All assigned exactly one shift): Valid.

RESULT:

Final Valid Schedule Assignment:

- **Doctor 1 (D1):** Morning Shift
- **Doctor 2 (D2):** Afternoon Shift
- **Doctor 3 (D3):** Night Shift

QUESTION 2: Grid Navigation using Best-First / A* Search

1. AIM

To determine the optimal path and total path cost for a robot navigating a 5x5 grid from Start (S) to Goal (G) using a heuristic-guided search.

2. OBJECTIVE

- To represent a 5x5 grid state space with movement constraints (Up, Down, Left, Right).
- To calculate the Manhattan Distance heuristic function h(n) = |xn - xg| + |yn - yg| for expanded nodes.
- To demonstrate step-by-step priority queue (OPEN list) updates, node expansions, and trace the path/cost to Goal (G).

3. THEORY

Search parameters and state representation:

- **State Space:** A grid of size 5x5 with coordinates (x, y) where x, y in {0, 1, 2, 3, 4}.
- **Start Node (S):** Located at (0, 0).
- **Goal Node (G):** Located at (4, 4).
- **Step Cost g(n):** 1 unit per movement.

- **Manhattan Distance Heuristic h(n):** $h(n) = |xn - xg| + |yn - yg|$
- **Evaluation Function f(n):** Combining path cost g(n) and estimated heuristic cost to goal h(n): $f(n) = g(n) + h(n)$.

4. ALGORITHM (A* Search Algorithm)

Algorithm 2: A* Search with Manhattan Distance

Algorithm: A-Star-Grid-Search(Grid, Start, Goal)
Input: Grid map, Start state (xs, ys), Goal state (xg, yg).
Output: Optimal path from Start to Goal and total path cost.

Procedure:
1. Initialize an empty min-priority queue named OpenList and an empty set named ClosedList.
2. Compute g(Start) = 0.
3. Compute h(Start) = |xs - xg| + |ys - yg|.
4. Set f(Start) = g(Start) + h(Start).
5. Insert Start node into OpenList ordered by its f-value.
6. While OpenList is not empty:
   a. Pop the node N with the lowest f(N) value from OpenList.
   b. If N is equal to Goal, reconstruct and return the optimal path and total cost g(N).
   c. Add node N to ClosedList.
   d. For each valid movement direction (Up, Down, Left, Right) to reach neighbor M:
      i. If M is an obstacle or M is present in ClosedList, ignore it.
      ii. Calculate tentative path cost: tentative_g = g(N) + 1.
      iii. If M is not in OpenList or tentative_g < g(M):
         - Set parent of M to N.
         - Set g(M) = tentative_g.
         - Compute h(M) = |xm - xg| + |ym - yg|.
         - Set f(M) = g(M) + h(M).
         - Insert or update M inside OpenList.
7. If OpenList becomes empty and Goal is not reached, return failure.

5. INTERMEDIATE STEPS & RESULT

Environment Setup:

- 5x5 grid indexed from (0,0) to (4,4).
- Start (S) = (0,0), Goal (G) = (4,4).
- Obstacles located at (1,1), (2,1), (3,1).

Step-by-Step Node Expansion & Priority Queue Updates:

- **Initialization:** Insert S(0,0) into OpenList. g(S)=0, h(S)=|0-4|+|0-4|=8, f(S)=8.
- **Iteration 1:** Pop S(0,0). Expand neighbors: Down (1,0) and Right (0,1).
  - Neighbor (1,0): g=1, h=|1-4|+|0-4|=7, f=8.
  - Neighbor (0,1): g=1, h=|0-4|+|1-4|=7, f=8.
  - OpenList: [(1,0) (f=8), (0,1) (f=8)].
- **Iteration 2:** Pop (1,0). Expand neighbors: (2,0) [since (1,1) is an obstacle].
  - Neighbor (2,0): g=2, h=|2-4|+|0-4|=6, f=8.
  - OpenList: [(2,0) (f=8), (0,1) (f=8)].
- **Subsequent Expansions:** The search continues evaluating nodes moving towards Goal (4,4), prioritizing minimal f(n) = g(n) + h(n) values while bypassing obstacles.

RESULT:

Optimal Path and Cost:

- **Optimal Path:** (0,0) -> (1,0) -> (2,0) -> (3,0) -> (4,0) -> (4,1) -> (4,2) -> (4,3) -> (4,4)
- **Optimal Path Cost:** 8 units (8 steps at cost 1 each)

QUESTION 3: Autonomous Rescue Robot in Partially Observable Environment

1. AIM

To analyze constraints, establish an online search solution strategy, and demonstrate AI foundational principles (agents, rationality, environment classification) for a disaster-response rescue robot.

2. OBJECTIVE

- To analyze environment constraints and explain their impact on real-time decision-making.
- To formulate an online search/Uniform Cost Search (UCS) dynamic path-planning algorithm.
- To define the agent PEAS architecture, rationality criteria, and environment characteristics.
- To provide logical justification for choosing Online Search / UCS under partial observability.

3. THEORY & DETAILED ANALYSIS

a) Constraint Analysis & Impact on Decision-Making:

- **Four-directional Movement:** Restricts movement options to 4 discrete actions per cell, requiring localized spatial planning.
- **Blocked Cells (Obstacles):** Prevents direct line-of-sight navigation; forces the agent to discover dead ends and re-route dynamically.
- **Limited Sensing Capability:** Prevents offline global path computation. The robot must interleaving search with execution (Online Search).
- **Variable Step Costs (Standard=1, Risky Zone=3):** Adds non-uniform edge weights, requiring cost-sensitive algorithms like Uniform Cost Search (UCS) over Simple BFS.
- **Dynamic Knowledge Update:** Requires maintainable local map state memory (explored vs. unexplored nodes) to prevent looping.

b) AI Concept Demonstration:

- **PEAS - Performance Measure:** Performance Measure: Maximize survivors reached, minimize total path cost, minimize risk exposure.
- **PEAS - Environment:** Environment: Grid of rooms with obstacles, risky zones, and unknown goal distance.
- **PEAS - Actuators:** Actuators: Motors for directional movement (Move Up, Down, Left, Right).
- **PEAS - Sensors:** Sensors: Proximity sensors / cameras sensing local adjacent cells.
- **Rationality:** The robot acts rationally by selecting actions that minimize expected total path cost based on its current, dynamically updated local knowledge.
- **Environment Classification:** Partially Observable, Dynamic/Sequential, Deterministic, Discrete.

4. ALGORITHM (Online Uniform Cost Search Agent)

Algorithm 3: Online Uniform Cost Search Agent

Algorithm: Online-Rescue-Agent-UCS(Start, Goal)
Input: Starting cell location Start, target survivor location Goal.
Output: Safe path traversed to Goal or failure notification.

Procedure:
1. Set Current_State = Start.
2. Initialize an empty table Visited_States to track visited coordinates and minimum costs.
3. Initialize an empty stack Path_History to record movement history for backtracking.
4. While Current_State is not equal to Goal:

a. Perception Step: Scan immediate adjacent cells (Up, Down, Left, Right) to detect obstacles and step costs.
b. Map Update: Add or update Current_State in Visited_States with the current path cost.
c. Identify Valid Actions: Filter adjacent cells to identify reachable, unblocked neighbors.
d. Decision Logic:
i. If all adjacent moves lead to visited states or obstacles (dead end):
- If Path_History is empty, exit and return failure (Goal unreachable).
- Pop previous state Backtrack_Node from Path_History.
- Execute movement action to return to Backtrack_Node.
ii. Otherwise:
- Select the unvisited adjacent action that minimizes cumulative step cost g(next_state).
- Push Current_State onto Path_History.
- Execute the chosen movement action.
e. State Update: Update Current_State to the newly reached cell coordinate.
5. Return Success when Current_State matches Goal.

5. RESULT & JUSTIFICATION

Justification of Chosen Approach:

- **Why Online Search?** Offline search (like standard A*) requires complete map knowledge before execution. Because the robot has limited sensing capabilities, offline search fails when encountering unmapped obstacles. Online search interleaves perception, planning, and execution.
- **Why Uniform Cost Search (UCS)?** Because entering risky zones incurs a higher cost (1 + 2 = 3) compared to normal steps (1), standard Breadth-First Search (BFS) would find the shortest path in terms of step count, not cost. UCS guarantees optimality for paths with non-uniform edge weights.